ConfidenceOS

The HMI That Knows What It Does Not Know

FINAL PROJECT DOCUMENTATION

Next-Generation Control System Interface

Read-only trust-aware HMI overlay, verification workflow, handover system, and model-driven Runtime generation platform.

Project Snapshot

Read-only
No process control writes

1 Hz
Live runtime stream

5 roles
Operator to auditor

4-factor
Confidence rubric

Studio
Generated Runtime flow

Audit
Ledger and evidence trail

Core idea

Traditional HMIs show values. ConfidenceOS shows values together with whether those values deserve trust, what evidence supports them, what decisions should be frozen, and what must be handed over.

1. Executive Summary

ConfidenceOS is a read-only, trust-aware Human-Machine Interface layer for industrial operations. It is designed for situations where operators cannot safely rely only on raw sensor values because a displayed value may be stale, miscalibrated, physically implausible, or contradicted by neighboring process data.

The project adds a trust layer beside the HMI/DCS. It reads live or replayed tag values, scores whether readings deserve trust, checks physical consistency using a mass-balance residual, identifies abnormal situations, creates field-verification tasks, preserves handover debt, and generates operator Runtime screens through an engineering Studio.

The outcome is an operator interface that does not simply display data. It explains which data can be used as operating basis, which data should not be trusted, what evidence replaces it, and what action must happen next.

> **Judging message**
> ConfidenceOS is not an alarm filter. It is a trust-aware operating layer that addresses the deeper problem of false certainty in industrial interfaces.

2. Problem Statement

Industrial operators make high-consequence decisions from HMI screens. However, most HMIs display values with equal visual confidence. A transmitter can drift, freeze, or be contradicted by process physics while the screen still presents the reading as a normal number.

This creates a dangerous gap between **what the system displays** and **what is physically true**. ConfidenceOS is built to close that gap.

```
Traditional HMI:
    Shows the sensor value.

ConfidenceOS:
    Shows the sensor value.
    Shows whether the value should be trusted.
    Shows why.
    Shows what decision should be frozen.
    Shows what verification is required.
    Preserves the issue for shift handover.
```

Incident Motivation

The project is motivated by industrial events where operators had data but not enough trust context. A level reading may look safe while mass balance says the vessel inventory is rising. A valve may show commanded state instead of actual state. A temperature may be displayed without enough context about whether that state is physically abnormal for the equipment.

ConfidenceOS turns those hidden uncertainty conditions into visible operator information.

3. Project Objectives

- Attach a clear trust state and confidence score to live sensor readings.

- Use calibration, stability, cross-sensor consistency, and physical plausibility as evidence.
- Detect physical contradictions using mass-balance residual checking.
- Translate technical evidence into operator-facing guidance.
- Generate field-verification tasks when trust cannot be established from screen data alone.
- Preserve unresolved operating debt across shift handover.
- Generate Runtime HMI screens from metadata, asset models, templates, and approved mappings.
- Keep the system read-only and separate from plant control authority.

4. Proposed Solution

ConfidenceOS sits beside existing control-system infrastructure as a read-only decision-support layer. It does not replace the DCS, SIS, historian, CMMS, or operator authority. It observes data, evaluates trust, and presents evidence.

Runtime

Operator-facing HMI screen that shows live values, trust state, abnormal situation, decision freeze, trusted substitutes, and single safe move.

Studio

Engineering workspace for tag import, Mapping Court, template assignment, validation, build receipts, and Runtime publication.

Work Queue

Maintenance and engineering workflow for field verification with structured evidence and lifecycle state.

Shift Channel

Handover console that preserves unresolved trust debt, verification tasks, notes, and operational trace references.

5. Feature Overview

Capability	What it delivers to the project
Trust-aware Runtime HMI	Every value is presented with a trust state, confidence evidence, and operating guidance.
Confidence Scoring Engine	Combines calibration age, stability, cross-sensor evidence, and physical plausibility into a transparent trust rubric.
Mass-Balance Residual Check	Uses conservation-of-volume logic to detect physically suspicious disagreement between level and flows.
Trust Quarantine	Prevents a contradicted sensor value from being treated as a valid decision basis.
Field Verification Workflow	Turns low-trust readings into actionable maintenance and engineering verification tasks.
Shift Channel	Preserves unresolved trust debt, verification state, notes, and operational context across shift handover.
Studio and HMI Compiler	Supports tag import, Mapping Court, template assignment, validation, build receipts, and published Runtime manifests.
Operational Ledger	Creates an auditable trail of events, notes, verification transitions, and handover-critical information.
Role-aware Workspaces	Gives operators, maintenance, engineers, managers, and auditors focused views of the same system state.

6. System Architecture

The architecture separates read-only data sources, runtime intelligence, engineering Studio, persistence, and frontend workspaces. This separation keeps the safety boundary clear while allowing the Runtime display to be generated from engineering metadata.

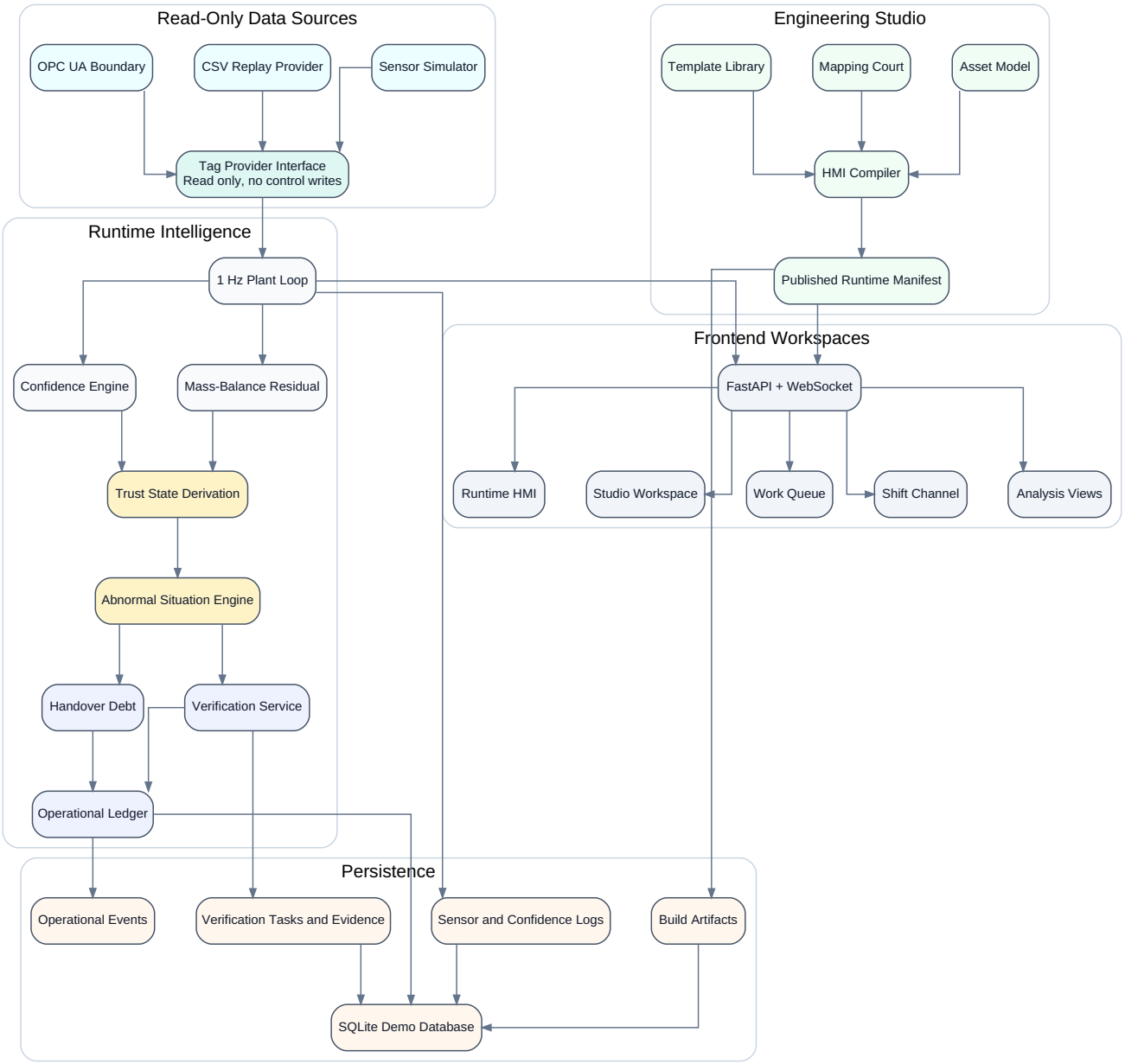

Figure 6.1 shows the ConfidenceOS architecture across read-only data sources, runtime intelligence, engineering Studio, persistence, and frontend workspaces.

7. Runtime Operating Flow

Runtime processing begins with live tag readings and ends with operator-facing decision support. The main value is not the confidence score alone, but the conversion of that score into trust state, operating basis, action, verification, and handover continuity.

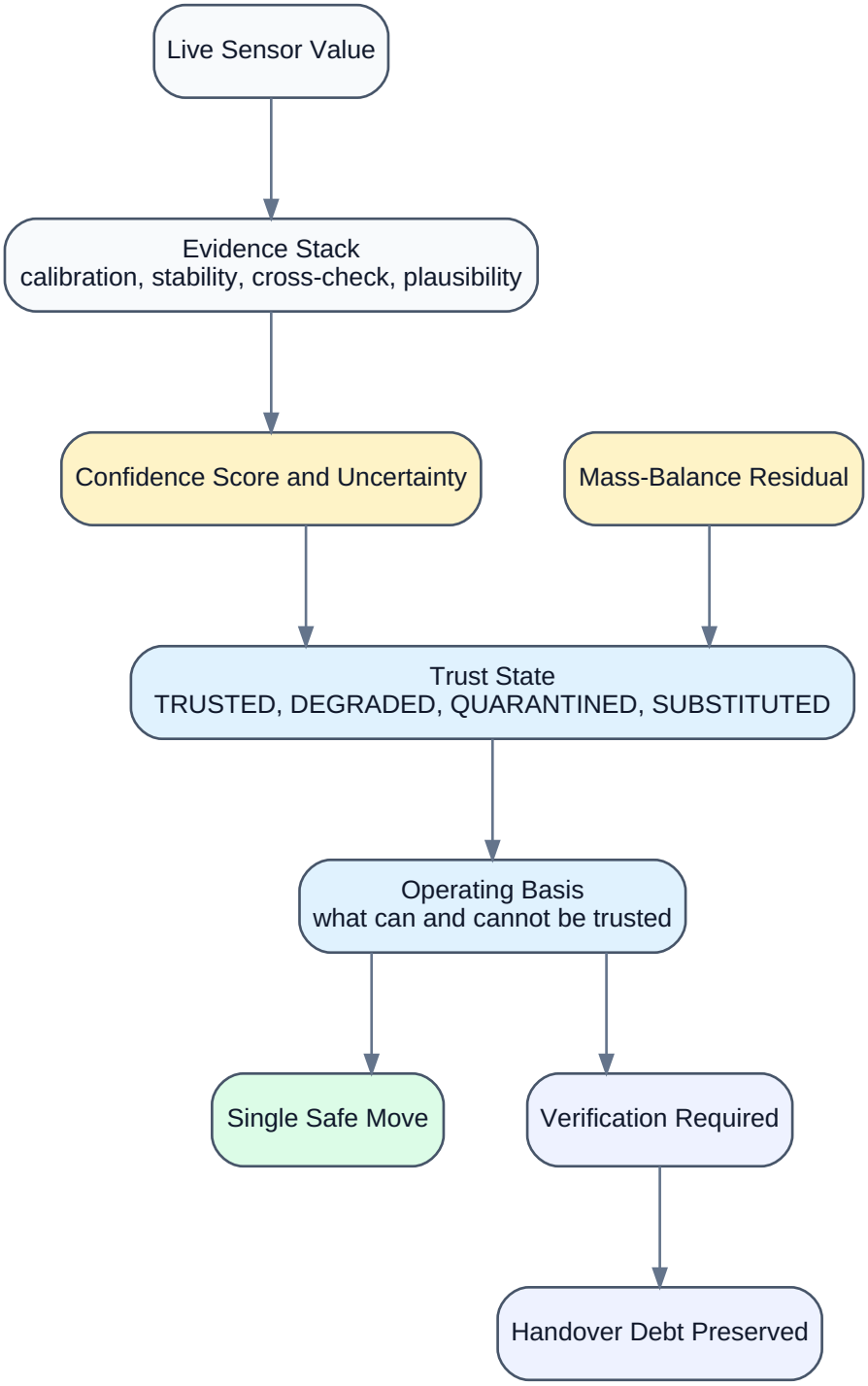

Figure 7.1 shows how ConfidenceOS converts live sensor values into trust state, operating basis, safe action, verification, and handover debt.

8. HMI Compiler and Studio Flow

Studio is the engineering side of ConfidenceOS. It helps convert raw tags and asset metadata into usable Runtime screens. The compiler flow ensures that unresolved or ambiguous tags are not silently published as operator-facing screens.

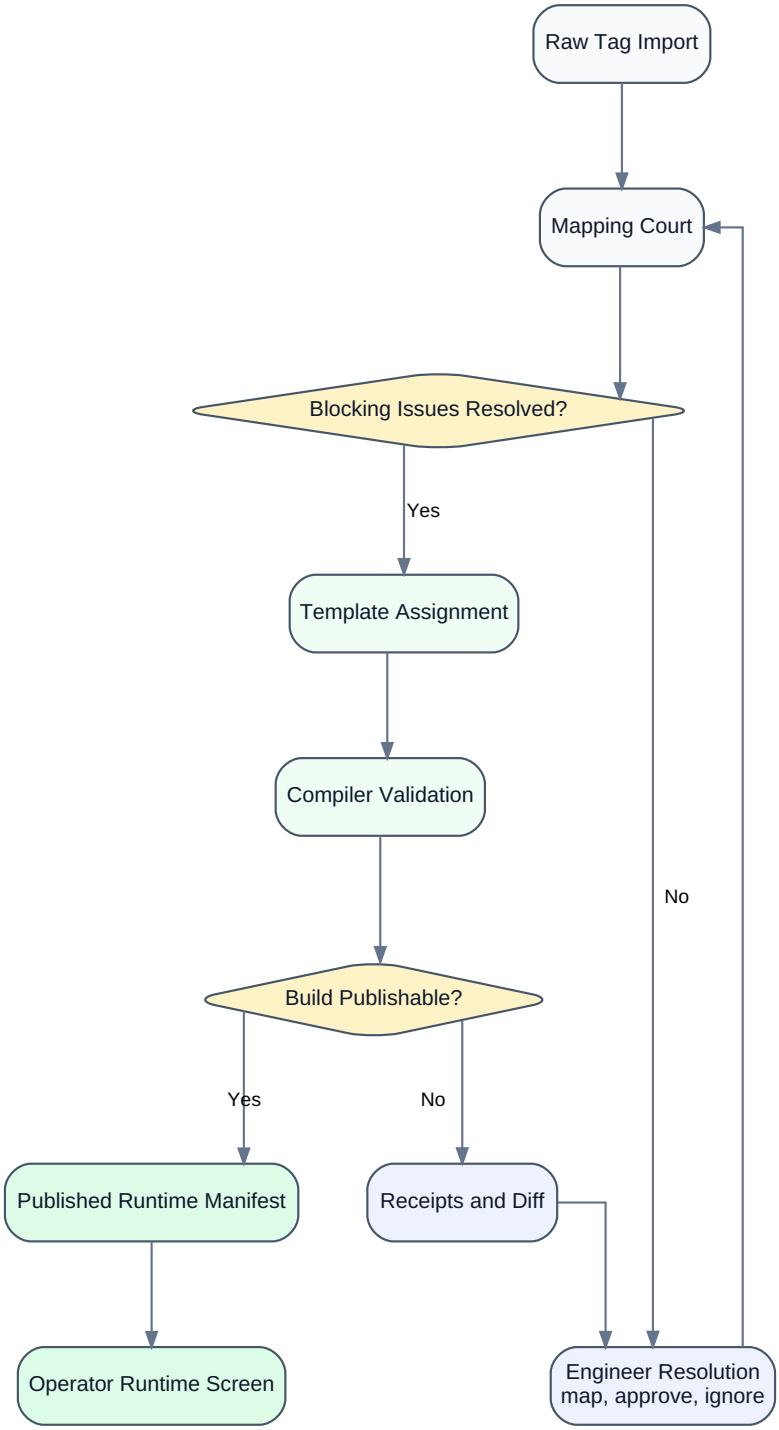

Figure 8.1 shows the Studio and HMI Compiler workflow from raw tag import to a published Runtime manifest.

9. Verification Workflow

When a value cannot be trusted from available evidence, ConfidenceOS creates a verification task. Maintenance performs the field check and captures evidence. Engineering or management can accept or reject the result. Every transition is designed to be auditable.

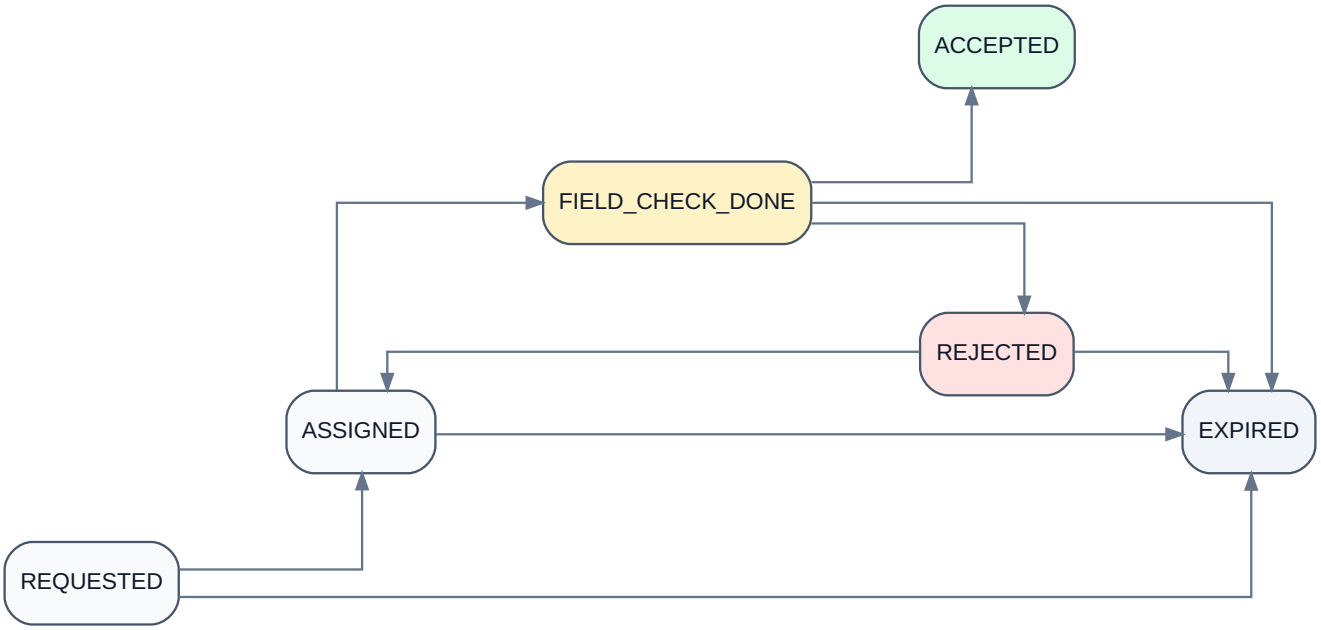

Figure 9.1 shows the field-verification lifecycle used to close the loop between operator uncertainty, maintenance action, and engineering acceptance.

10. Role-Based Product Experience

ConfidenceOS provides different experiences for different industrial users. Operators receive simplified decision guidance; maintenance receives actionable field tasks; engineers receive configuration and acceptance tools; managers and auditors receive review and traceability views.

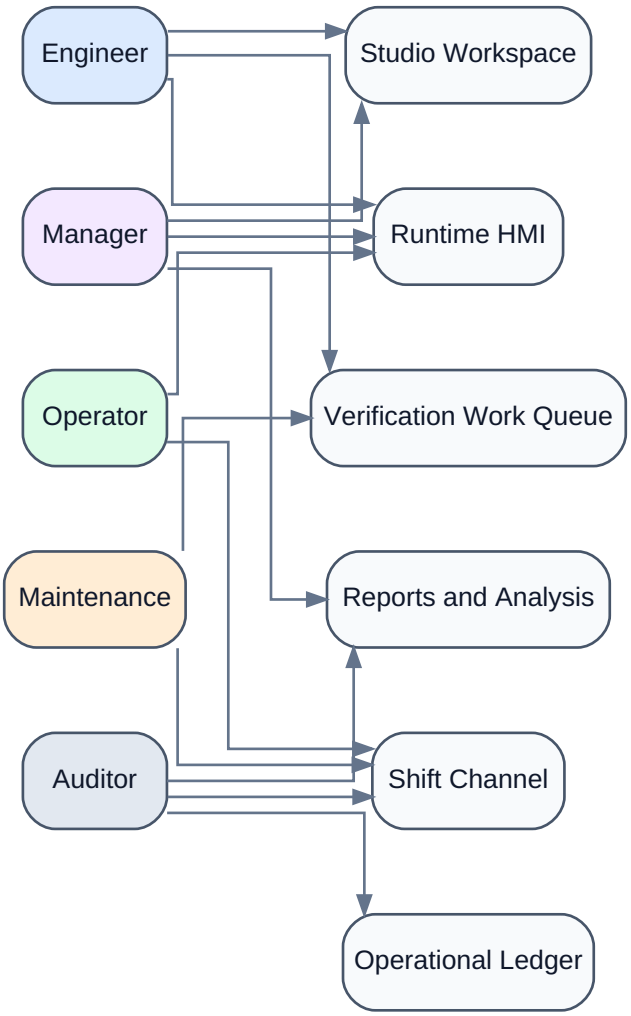

Figure 10.1 shows how user roles map to ConfidenceOS workspaces and responsibilities.

11. Core Algorithms

11.1 Confidence Scoring

The confidence engine is a governed trust rubric. It is not presented as a statistical probability. It combines calibration age, signal stability, cross-sensor consistency, and physical plausibility to create a readable trust score and evidence stack.

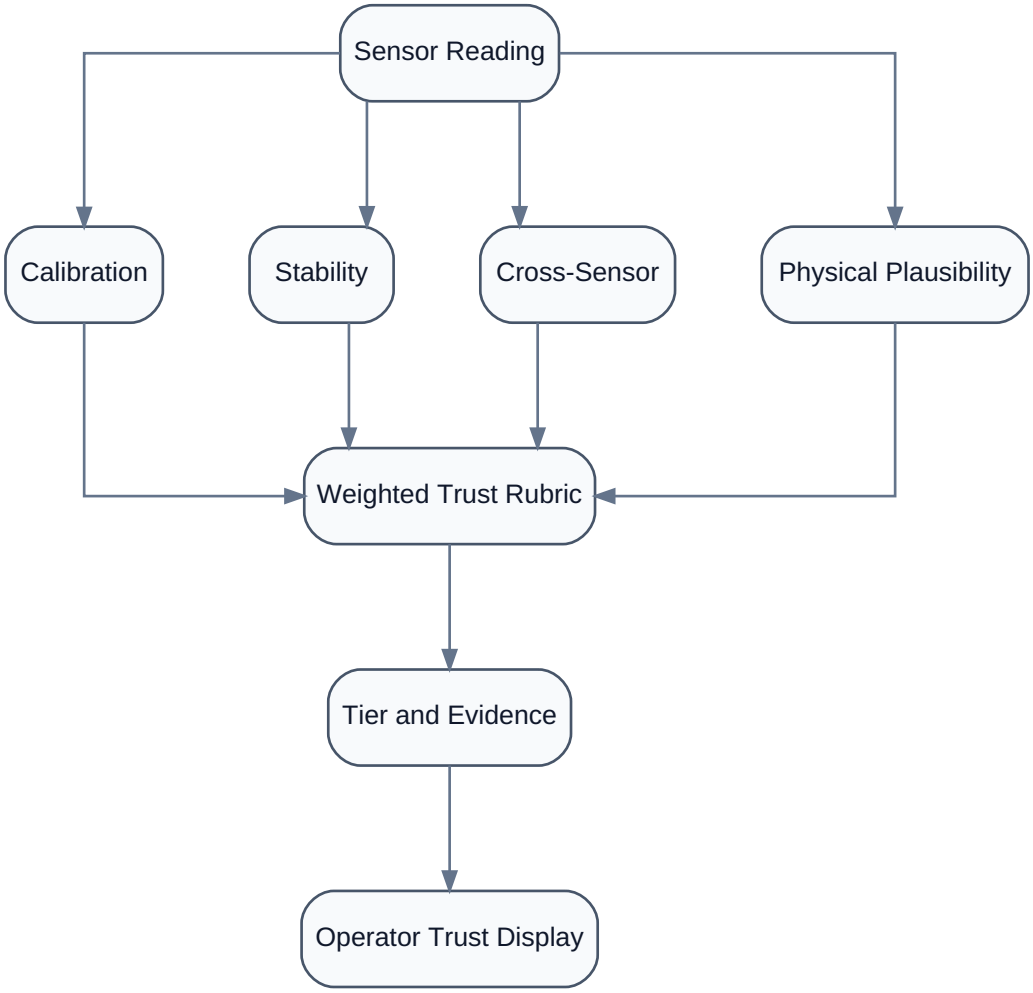

Figure 11.1 shows the confidence scoring algorithm from sensor reading to operator trust display.

```
confidence(sensor) =
    0.30 * calibration_score
  + 0.20 * stability_score
  + 0.30 * cross_sensor_score
  + 0.20 * physical_plausibility_score

Output tiers:
    HIGH      80-100
    MEDIUM    50-79
    LOW       20-49
    CRITICAL   0-19
```

11.2 Mass-Balance Residual

The mass-balance engine compares what inflow and outflow imply against what the level sensor reports. It is implemented as a single-vessel volumetric residual check, which is appropriate for demonstrating physical contradiction while keeping assumptions clear.

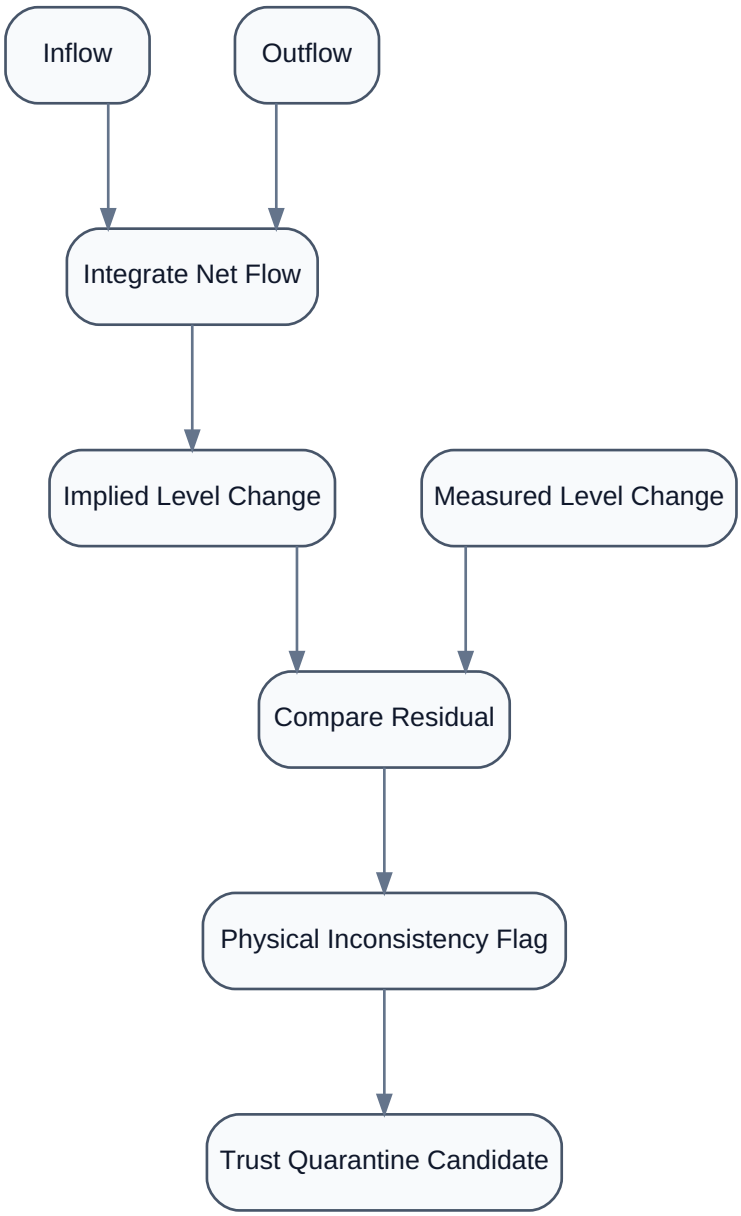

Figure 11.2 shows the mass-balance residual method used to detect physical inconsistency between flow-derived and measured level change.

```
implied_level_delta = integral(inflow_rate - outflow_rate, dt) * flow_to_level_rate
measured_level_delta = current_level - level_at_window_start
discrepancy = abs(implied_level_delta - measured_level_delta)
```

12. Functional Modules

Module	Responsibility
Sensor Simulator and Replay	Produces realistic industrial tag streams for level, flow, pressure, temperature, valve, and demo replay data.
Confidence Engine	Calculates score, tier, evidence, uncertainty, NAMUR-style condition language, and recommended action.
Mass-Balance Engine	Computes residual between flow-implied and measured level change.
Advisory Engine	Converts degraded trust and physical contradictions into abnormal situations and operator guidance.
Verification Service	Manages durable task lifecycle, procedure guidance, evidence, and audit events.
Studio Service	Maintains engineering state, Mapping Court, builds, published manifests, and build artifacts.
Shift Channel	Aggregates handover debt, notes, verification tasks, and operational ledger references.
Frontend Runtime	Presents the operator workplace, trust state, decision freeze, and single safe move.
Frontend Studio	Presents the engineering configuration and compiler workflow.
Analysis Views	Provide fleet integrity, degradation forecasts, forensics, causal graph, compliance, and engineer detail views.

13. User Interface and Workspaces

The user interface is organized around primary industrial workflows rather than generic dashboards. Runtime is the operator workplace. Studio is the engineering workspace. Work Queue is the field verification console. Shift Channel is the handover and operating-debt board.

Runtime HMI

- Live process values with trust state.
- Abnormal situation summary.
- Trust Quarantine and trusted substitutes.
- Decision freeze and single safe move.
- Mass-balance and confidence evidence.
- Read-only boundary messaging.

Studio Workspace

- Raw tag import and Mapping Court.
- Asset model and signal review.
- Template assignment and validation.
- Compiler build, receipts, publish readiness, and Runtime publication.

Work Queue

- Verification task status.
- Procedure-specific evidence requirements.

• Assignment, field check completion, acceptance, rejection, and expiry.

Shift Channel

• Handover blockers and unresolved trust debt.
• Verification tasks and evidence context.
• Operator notes and operational ledger references.

14. Technology Stack

Layer	Technology Used	Purpose
Backend API	FastAPI, Uvicorn	REST and WebSocket services for live runtime data and workflows.
Language	Python 3.11	Simulation, scoring, mass-balance, verification, and compiler logic.
Persistence	SQLite, SQLAlchemy	Demo persistence for readings, confidence logs, tasks, evidence, ledger, and build artifacts.
Frontend	React 19, Vite 8	Role-aware Runtime, Studio, Work Queue, Shift Channel, and analysis views.
State and Charts	Zustand, Recharts	Live WebSocket state and visualization of trends and process evidence.
AI Assistance	Anthropic SDK with fallback behavior	Advisory explanations, handover text, and mapping support when configured.
Styling	Tailwind CSS 4, custom design tokens	Industrial HMI-style UI with disciplined status presentation.
Deployment	Docker-ready backend and Vite build	Demo-friendly deployment with API and WebSocket routing.

15. Database Design

The persistence layer stores readings, confidence history, alarms, verification tasks, evidence, audit events, operational ledger events, Studio state, build artifacts, and users. This allows the system to support both live display and traceability.

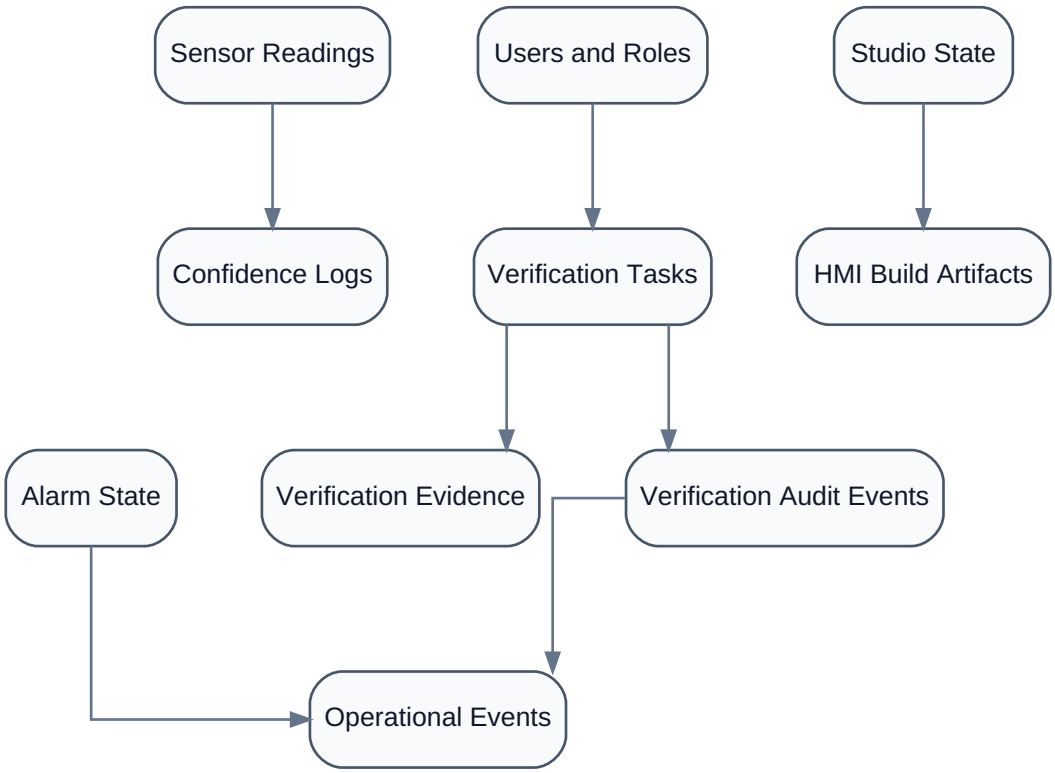

Figure 15.1 shows the main persistence entities used by ConfidenceOS.

16. API and Integration Overview

The backend exposes REST endpoints for runtime state, confidence, mass balance, generated screens, Studio, verification, handover, fleet integrity, predictions, and operational ledger. The live Runtime data stream is provided through WebSocket.

Area	Representative Interfaces
Live Runtime	/ws/sensors, /api/sensors/latest, /api/confidence, /api/mass-balance/state
Generated Screens	/api/screens/generated, /api/runtime/navigation, /api/runtime/situations
Studio	/api/studio, /api/studio/mapping-court, /api/studio/build/run, /api/studio/publish
Verification	/api/verification-tasks, /api/verification-tasks/audit, /api/verification-tokens
Handover	/api/shift-channel, /api/shift-channel/note, /api/handover/debt
Analysis	/api/fleet, /api/fleet/history, /api/predictions, /api/operational-ledger

17. ABB Challenge Alignment

Evaluation Area	ConfidenceOS Response
Problem clarity	Targets false certainty in HMI readings rather than only alarm count.
Innovation	Adds explicit trust state, Trust Quarantine, substitute evidence, and handover debt to HMI operation.
Industrial relevance	Uses instrumentation concepts such as calibration, stability, cross-sensor consistency, mass balance, field verification, and operator handover.
Model-based engineering	Studio turns asset models, tags, templates, mappings, validation, and receipts into generated Runtime screens.
Role awareness	Operators see decision guidance, maintenance sees field tasks, engineers see mappings and builds, auditors see traceability.
Safety posture	System remains read-only and does not replace DCS, SIS, or operator authority.

18. Safety and Read-Only Boundary

ConfidenceOS is deliberately designed as a read-only layer. It does not send control commands, change setpoints, operate valves or pumps, acknowledge plant alarms in the control system, or replace safety systems.

```
ConfidenceOS reads process data.
ConfidenceOS computes trust, evidence, and operating guidance.
ConfidenceOS displays decision-support information.
ConfidenceOS does not control the plant.
```

This boundary is important because the product is meant to increase operator awareness without creating a new unsafe control path.

19. Testing and Validation Approach

The validation approach focuses on proving that the system behaves coherently across roles and workflows.

- Verify login and role-based navigation for all roles.
- Verify Runtime loads and live WebSocket updates are visible.
- Verify confidence score changes under simulated failure modes.
- Verify mass-balance divergence creates visible operator evidence.
- Verify Trust Quarantine changes operating-basis guidance.
- Verify verification tasks move through legal lifecycle states.
- Verify evidence is required before acceptance.
- Verify Studio blocks publish when mappings are unresolved.
- Verify Shift Channel preserves unresolved operating debt.
- Verify ledger and audit views expose traceable events.

20. Current Boundaries and Future Scope

The current implementation is a strong project prototype. It is intentionally honest about what is simulated, what is heuristic, and what would require additional production engineering.

Current Boundary	Future Enhancement
SQLite is used for demo persistence.	Move to PostgreSQL or an industrial time-series/historian-backed architecture.
OPC UA is prepared as a read-only boundary, not production-connected by default.	Add secure subscriptions, certificates, namespace mapping, quality flags, and reconnect handling.
Mass balance is single-vessel and volumetric.	Support multi-unit, multi-stream, density-aware, delay-aware process models.
Confidence score is a governed trust rubric.	Calibrate against plant history, maintenance records, and validated instrumentation data.
AI is advisory.	Keep AI bounded, explainable, and subordinate to deterministic validation.
JWT auth is suitable for demonstration.	Integrate enterprise identity, SSO, RBAC, and signed audit trails.

21. Expected Impact

For Operators

Faster recognition of what is wrong, what not to trust, and what action is safe.

For Maintenance

Clearer field tasks, procedure-specific evidence, and closure criteria.

For Engineers

Repeatable tag mapping, template assignment, validation, build receipts, and published Runtime screens.

For Managers

Better visibility into unresolved operating debt and handover readiness.

For Auditors

Traceable verification, handover, and operational events.

For Industrial Software Evaluation

A clear next-generation HMI concept that combines operator UX, engineering workflow, and trust-aware runtime intelligence.

22. Conclusion

ConfidenceOS reframes the purpose of an industrial HMI. A traditional HMI shows values. ConfidenceOS shows values, trust, evidence, consequences, verification needs, and handover obligations.

The project combines confidence scoring, physical residual checking, Trust Quarantine, generated Runtime screens, role-aware workflows, structured verification, Shift Channel continuity, and operational auditability. It remains read-only and respects the boundary between advisory intelligence and process control.

ConfidenceOS is the missing trust layer beside the control system.